Lecture # 8

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Attribution

TODAY'S LECTURE IS BASED ON PARALLEL RANDOM-ACCESS MACHINES BY MARC MORENO MAZA

Agenda

PRAM – Parallel Random-Access Machines

Algorithmics and Algorithm Engineering

Algorithmics and Algorithm Engineering

- **Algorithmics:** It's a subdiscipline of computer science dedicated to developing efficient algorithms. Algorithms are step-by-step procedures or formulas for solving problems, executing tasks, or processing data.
- **Algorithm Engineering (AE):** AE is a methodology within algorithmic research that emphasizes not only the design and analysis of algorithms but also their practical implementation and experimental evaluation. It's a cyclical process aimed at driving algorithmic research forward.

Introduction to Algorithmics and AE

Evolution: Historically, algorithmics was primarily theoretical, focusing on proving the correctness and efficiency of algorithms on paper. However, the advent of algorithm engineering shifted the focus towards practical implementation and experimentation.

Importance: AE plays a crucial role in bridging the gap between theoretical algorithms and real-world applications by emphasizing practical implementation, performance analysis, and experimentation.

Components of Algorithm Engineering

- **Design, Analysis, Implementation, and Experimental Evaluation**: These are the core components of AE, forming a feedback cycle where algorithms are designed, analyzed, implemented, and experimentally evaluated to drive further research.
- **Realistic Models and Algorithm Libraries**: AE relies on realistic models of computation and algorithm libraries to provide accurate descriptions, analysis, and implementations of algorithms.

Models in Parallel Computing

Shared Memory Machines:

- **Definition**: Shared memory machines allow all processors to access the same global memory, simplifying communication and data sharing among processors.
- **Challenges**: Specifying memory access times accurately can be challenging due to contention issues when multiple processors access the same memory simultaneously.

Distributed Memory Machines:

- **Definition**: Distributed memory machines consist of multiple sequential computers connected via an interconnection network, each with its own local memory.
- **Models**: Models like LogP and Bulk Synchronous Parallel (BSP) provide simplified representations of distributed memory systems, aiding in algorithm design and analysis.

Designing Parallel Algorithms

Efficiency vs. Simplicity:

- Parallel algorithms must balance efficiency with simplicity. While complex algorithms may be highly efficient, they can be challenging to implement and maintain.
- Example: Parallelizing a sorting algorithm like Quick Sort involves designing efficient partitioning schemes while keeping the implementation manageable.

Analysis of Parallel Algorithms

Challenges in Analysis:

- Analyzing the performance of parallel algorithms can be challenging due to factors like communication overhead, load balancing, and scalability.
- Example: Analyzing the performance of parallel matrix multiplication algorithms requires considering factors like data distribution, communication patterns, and computational complexity.

Random Access Machines

The Random Access Machine is a convenient model of a sequential computer. Its features are as follows.

- The computation unit executes a user defined program.
- It uses a read-only input tape and a write-only output tape.
- The RAM has an unbounded number of local memory cells.
- Each memory cell can hold an integer of unbounded size.
- Execution starts with the first instruction of the program and terminates when a Halt instruction is reached

Random Access Machines

Each operation takes one time unit regardless of the operand sizes.

Time complexity = the number of instructions executed.

Space complexity = the number of memory cells accessed.

Implementation Challenges

Semantic Gaps:

- Translating abstract algorithmic concepts into efficient parallel implementations can be challenging due to semantic gaps between algorithmic descriptions and programming languages.
- Example: Implementing parallel algorithms using low-level constructs like threads or message passing requires understanding the underlying hardware architecture.

Performance Tuning:

- Achieving optimal performance on parallel hardware involves systematic performance tuning to minimise communication overhead, maximize parallelism, and optimize resource utilization.
- Example: Tuning parameters like thread affinity, cache locality, and load balancing can significantly impact the performance of parallel applications.

Parallel Random-Access Machines (PRAMs)

PRAMs

- **What are PRAMs?**
  - Abstract model for parallel computation.
  - Assume synchronous operation of processors accessing shared memory.
- **Why PRAMs?**
  - Generalize parallel computation from the RAM model.
  - Facilitate analysis of parallel algorithms.

Understanding PRAM (Parallel Random-Access Machine)

- **Introduction:**
  - PRAM allows multiple processors to access memory simultaneously for efficient parallel processing.
  - It assumes a shared memory architecture with multiple processors.
- **Key Concepts:**
  - Synchronous operation: All processors operate in lockstep.
  - Shared memory: Processors access a common memory space.

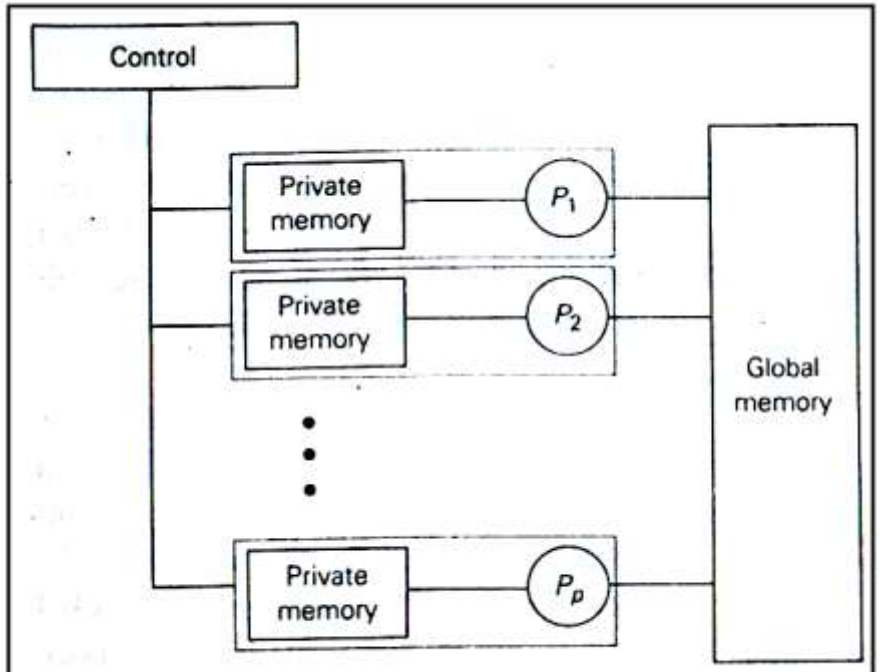


Figure 4.1 PRAM model for parallel computations

PRAM Operations

- **Read (R):**
  - A processor reads a value from a memory location.
  - Concurrent reads are allowed.
- **Write (W):**
  - A processor writes a value to a memory location.
  - Exclusive write or concurrent write models are possible.
- **Example:**
  - Processor P1 reads from memory location A.
  - Processor P2 writes to memory location B.

Parallel Random Access Machine

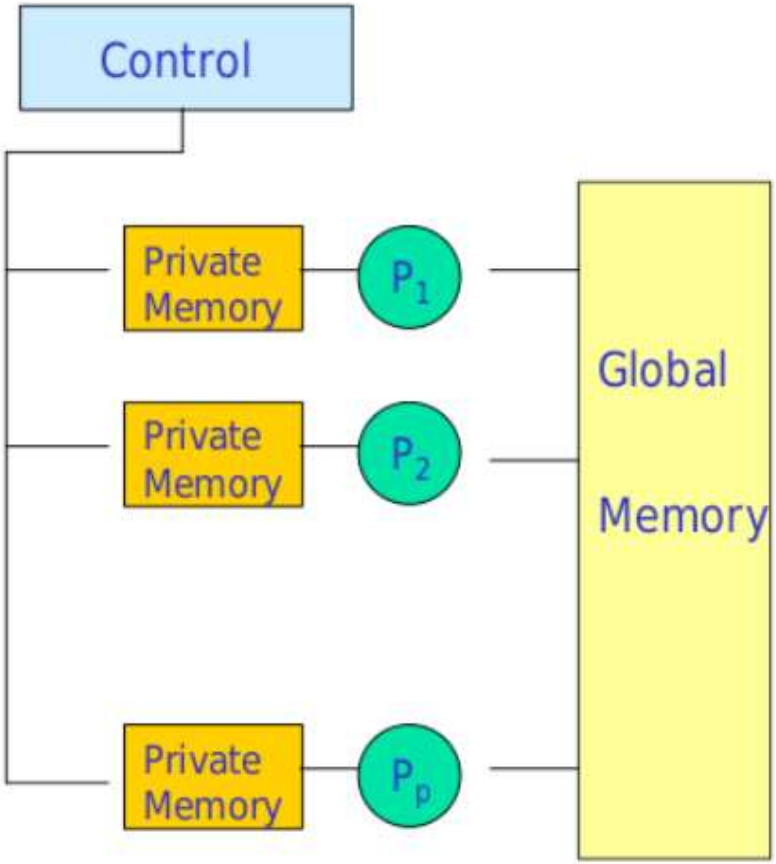
The Parallel Random Access Machine is a natural generalization of RAM.

- It is also an idealization of a shared memory machine.

Its features are as follows.

- It holds an unbounded collection of RAM processors $P0$, $P1$, $P2$, ... without tapes.
- It holds an unbounded collection of shared memory cells $M[0]$,$M[1]$,$M[2]$, . . .
- Each processor Pi has its own (unbounded) local memory (register set).
- Each processor Pi can access any shared memory cell $M[j]$ in unit time, unless there is a conflict.

PRAM


Control
Private Memory
P_1
Private Memory
P_2
Private Memory
P_p
Global Memory

PRAM Model

The input of a PRAM program consists of n items stored in $M[0], \ldots, M[n-1]$.

The output of a PRAM program consists of n' items stored in n' memory cells, say $M[n], \ldots, M[n+n'-1]$.

A PRAM instruction executes in a 3-phase cycle:

- Read (if needed) from a shared memory cell,
- Compute locally (if needed),
- Write in a shared memory cell (if needed).

All processors execute their 3-phase cycles synchronously.

Special assumptions must be made to resolve shared memory access conflicts.

The only way processors can exchange data is by writing into and reading from memory cells.

PRAM Model

$P0$ has a special activation register specifying the maximum index of an active processor:

- Initially, only $P0$ is active; it computes the number of required active processors
- Then, $P0$ loads this number in the activation register,
- The corresponding processors start executing their programs.

Computations proceed until $P0$ halts, at which time all other active processors are halted.

Question

WHAT IS THE TIME AND SPACE COMPLEXITY OF THE PRAM?

PRAM Model's Complexity

Parallel space complexity = the maximum number of shared memory cells in use during the computations.

Parallel time complexity = the time for $P0$'s computations.

Alternatively, if special activation register is not used i.e. all processors are available from start, the time complexity will be of the longest running processor.

Why use PRAM

The PRAM Model is attractive for designing parallel algorithms:

It is natural: the number of operations executed per one cycle on p processors is at most p.

It is strong: any processor can read or write any shared memory cell in unit time.

It is simple: ignoring any communication or synchronization overhead.

This natural, strong and simple PRAM model can be used as a benchmark: If a problem has no feasible (or efficient) solution on a PRAM then it is likely that it has no feasible (or efficient) solution on any parallel machine.

Example 1

Parallel search of an item x

- in an unsorted input file with n items,
- in a shared memory with p processors,
- where any cell can be accessed by only one processor at a time.

Broadcasting x costs $O(\log(p))$, leading to

$$T(n, p) = O(\log(p)) + O(n/p).$$

Performance Counters

The **parallel efficiency**, denoted by $E(n, p)$, is

$$E(n, p) = \frac{SU(n)}{pT(n, p)}$$

where $SU(n)$ is a lower bound for a sequential execution.

Observe that we have $SU(n) \leq p\ T(n, p)$ and thus $E(n, p) \leq 1$.

One also often considers the **speedup factor** defined by:

$$\mathrm{S}(n, p) = \frac{SU(n)}{T(n, p)}$$

Performance Counters

A **data-processor iso-efficiency function** is an asymptotically maximal function $f1$ such that for all $p0 > 0$ there exists $n0$ such that for $n \geq n0$ we have

$$E(n, f1(n)) \geq E(n0, p0)$$

A **processor-data iso-efficiency function** is an asymptotically minimal function $f2$ such that for all $n0 > 0$ there exists $p0$ such that for $p \geq p0$ we have

$$E(f2(p), p) \geq E(n0, p0).$$

Performance Counters

The data-processor iso-efficiency function $f1$ quantifies the growth rate of the number of processors p required to keep the efficiency $E(n,p)$ fixed as the problem size n increases.

It reflects the ability of a parallel algorithm to effectively utilize additional processors without losing efficiency.

A large data-processor iso-efficiency function $f1$ indicates poor scalability, meaning that the parallel algorithm requires a disproportionately large number of processors to maintain efficiency as the problem size grows.

- In contrast, a small iso-efficiency function suggests that the algorithm can efficiently exploit newly added processors with only a modest increase in the problem size.

Performance Counters

The iso-efficiency function $f2$ quantifies the growth rate of the problem size, required to keep the efficiency fixed while increasing the number of processors.

It reflects the ability of a parallel algorithm to maintain a constant efficiency.

A large iso-efficiency function $f2$ indicates poor scalability

- In contrast, a small one indicates that only a small increment in the problem size is sufficient for efficient exploitation of newly added processors.

Example 2

Consider the following problem: summing n numbers on a small PRAM with $p \leq n$ processors.

Each processor adds locally $\frac{n}{p}$ numbers.

Then the p partial sums are summed using a parallel binary reduction on p processors in $\lceil \log(p) \rceil$ iterations.

Thus, we have: $T(n, p) \in O(\frac{n}{p} + \log(p))$

Therefore, we have $E(n, p) = \frac{n}{\log(p)p+n}$

Example 3: List Ranking with Pointer Jumping

A linked list of n objects

- Doubly-linked list

We want to compute the distance of each element to the end of the list

The sequential solution

- Iterate through the list from the end to the beginning
- Assign each element a distance from the last element while iterating
- This solution has a complexity (execution time) in O(n)

Can we do better with a parallel algorithm?

Example 4: Prefix Sum

The prefix sum or cumulative sum of a sequence of numbers x0, x1, x2, ... is a second sequence of numbers y0, y1, y2, ..., the sums of prefixes (running totals) of the input sequence:

- y0 = x0
- y1 = x0 + x1
- y2 = x0 + x1+ x2

Serial solution has a complexity (execution time) in O(n)

Can we do better with a parallel algorithm?

PRAM Variations

There are several variations of the PRAM model, each with different assumptions about how processors interact with the memory

Exclusive Read, Exclusive Write (EREW) PRAM

Concurrent Read, Exclusive Write (CREW) PRAM

Concurrent Read, Concurrent Write (CRCW) PRAM

How powerful are the different models:

CRCW > CREW > EREW

A model is more powerful if there is one problem for which this model allows implementing a strictly faster solution with the same number of PUs

CRCW Conflict Resolution Strategies

Concurrent writes can lead to conflict. Conflicts can be resolved using:

- **Arbitrary:** One of the writing processors succeeds (unpredictable).
- **Priority:** The processor with the highest (or lowest) ID succeeds.
- **Common:** All processors must write the same value; otherwise, the write fails.
- **Combining:** Values written are combined using a specific operation (e.g., sum, product, max).

Example 5

Reduce sum with CRCW model

Attribution

COMING PART OF LECTURE IS BASED ON PARALLEL PROGRAMMING USING SKELETON PATTERNS BY DARLINGTON ET. AL

Agenda

PRAM (contd.)

Parallel Skeleton

Parallel Skeletons

Many parallel applications share common internal interaction patterns.

- For example, image processing algorithms can often be described as pipelines

A parallel skeleton is a programming construct (or a function in a library), which abstracts a pattern of parallel computation and interaction.

Programmers provide application-specific operations

- The skeleton handles parallel execution, composition, and optimization.

Why use Parallel Skeleton

Skeleton programming offer certain benefits:

Abstraction: Hides the low-level details of parallel execution.

Productivity: Reduces development time and effort.

Correctness: Skeletons are typically proven correct, reducing the risk of errors.

Portability: Can be implemented on different parallel platforms.

Example: A pipeline skeleton

Programmer responsibility

Define the computation for each stage of the pipeline.

Skeleton responsibility

Scheduling stages onto processors.

Handling inter-stage communication.

Example: Divide and conquer skeleton

Programmer responsibility

Define the operation to divide a problem.

Define the operation to combine sub-solutions.

Define the condition for problem divisibility.

Define the method to solve indivisible problems.

Skeleton responsibility

Deciding whether to use parallelism.

Managing dynamic scheduling.

Handling granularity of parallelism.

Managing inter-process interaction.

Classification of Parallel Skeletons

There are two broader classes of parallel skeletons:

Data-parallel skeletons are higher-order functions operating on bulk data structures, providing a functional approach to parallel programming.

Task and algorithm-oriented skeletons abstract parallel computation by encapsulating control structures and algorithmic paradigms

Common Data Parallel Skeletons

Map $map\ f[x_1 \dots x_n] = [f x_1 \dots f x_n]$

- Map is the simplest – and at the same time the "most parallel" – functional, which applies a unary function f to each element of a list

Reduce $red(\oplus)[x_1 \dots x_n] = x_1 \oplus x_2 \dots \oplus f x_n]$

- Reduction can be computed in parallel in a tree-like manner with logarithmic time complexity, owing to the associativity of the base operation

Scan $scan(\oplus)[x_1 \dots x_n] = x_1, x_1 \oplus x_2 \dots, x_1 \oplus \cdots \oplus f x_n]$

- Scan functional can be computed with logarithmic time complexity, despite an apparently sequential data dependence between the elements of the resulting list

Common Task Parallel Skeletons

Farm can be viewed as the simplest task-parallel skeleton.

Farm is effectively equivalent to map, calling for some operation to be applied independently to each component of a bulk data structure.

- However, the use farm of farm implies the execution cost of these applications is unpredictable and variable
- The data can be sourced from a stream or an array
- Typically uses master-worker approaches

Common Task Parallel Skeletons

Pipeline skeletons capture the pattern in which a stream of data is processed by a sequence of “stages”

- Performance is derived from parallelism both between stages, and where applicable, within stages

Pipeline skeletons governs internal decisions including

- the selection of the number of implementing processes or threads
- allocation of operations to processes or threads, whether statically or dynamically
- correct choice of synchronization mechanism.

Common Task Parallel Skeletons

Divide-and-conquer skeleton recursively break down problems into sub-problems, solve them, and combine the solutions.

Skeleton design involves trade-offs between API flexibility and implementation optimizations.

- A generic API requires "divide," "conquer," a recursion test, and a direct solution method.
- Less flexible APIs might fix the division degree, recursion depth, or constrain "divide" and "conquer" operations.

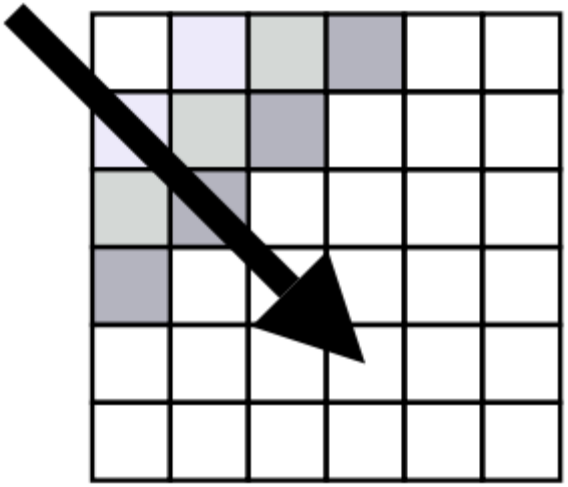
Example

Merge sort

Common Task Parallel Skeletons

Wavefront skeleton are useful for task where computation are performed as diagonal sweep, where the next computation relies on previous computation

A wavefront skeleton may require the programmer to describe the stencil and the operation performed within it, leaving the implementation to determine and schedule an appropriate level of parallelism.



Often involves dynamic programming